

Network Administration / System Administration

Homework 1

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

1. Discussion

- 這次作業我和 賴泓安、鄭竣陽 以及 喻慈恩 討論。

2. Reference

Problem 1. 參數檢查

- [How can I have a newline in a string in sh? \(Stack Overflow\)](#)
- [How can I match a string with a regex in Bash? \(Stack Overflow\)](#)
- [正規表達式 - wikipedia](#)
- [How do I use a regex in a shell script? \(Stack Overflow\)](#)
- [Regular Expression 正則表達式初探 \(TimCodingBlog\)](#)
- [test man page](#)

Problem 2. Building a Merkle Tree

- [How can I store the "find" command results as an array in Bash \(Stack Overflow\)](#)
- [Add a new element to an array without specifying the index in Bash \(Stack Overflow\)](#)
- [Loop through an array of strings in Bash? \(Stack Overflow\)](#)
- [Convert absolute path into relative path given a current directory using Bash \(Stack Overflow\)](#)
- [test man page](#)

Problem 3. Generating an Inclusion Proof

- [Bash bitwise operators - xor \(Sysxplore\)](#)

Problem 4. Verifying an Inclusion Proof

- [Using multiple delimiters in awk \(Stack Overflow\)](#)
- [Bash bitwise operators - and \(Sysxplore\)](#)
- [How to convert a string to lower case in Bash \(Stack Overflow\)](#)

3. AI conversation

Problem 1. 參數檢查

Does bash script function have define order?

推理關於 bash 函式定義順序花了 16 秒 >

Bash does not support automatic “hoisting” of function definitions like some other languages (e.g., JavaScript). This means that if you try to call a function before its definition has been read and executed by the shell, you’ll run into an error. In practice, as long as your functions are defined (i.e. the code that creates them has been executed) before you call them, the order of definitions in the script does not matter.

For example, consider this script:

```
bash 複製 編輯  
  
#!/bin/bash  
  
# Calling the function before it is defined will cause an error.  
# greet # Uncommenting this line will lead to an error.  
  
# Define the function.  
greet() {  
    echo "Hello, world!"  
}  
  
# Now calling the function works fine.  
greet
```

In this script, the function `greet` must be defined before any code that calls it is executed. A common best practice is to place all function definitions at the beginning of your script (or in a separate file that you source) to avoid any ordering issues.

Problem 2. Building a Merkle Tree

本題我沒有使用 chatGPT 輔助

Problem 3. Generating an Inclusion Proof

我要怎麼一行一行讀取一個檔案直到空行，然後改為將接下來每一行的資料一行一行按照：分開使用 bash

你可以使用 `while read` 來逐行讀取檔案，並在遇到空行 ("") 後，將接下來的每一行按照：分割。

解法：

```
bash
#!/bin/bash

file="your_file.txt"
reading_first_part=true # 用來標記是否在第一部分

while IFS= read -r line; do
    if [ -z "$line" ]; then
        reading_first_part=false
        continue # 跳過空行
    fi

    if $reading_first_part; then
        echo "First part: $line"
    else
        IFS=":" read -r first second <<< "$line"
        echo "First: $first, Second: $second"
    fi
done < "$file"
```

說明：

1. `reading_first_part=true` 來標記是否仍在第一部分。
2. 遇到空行 (`[ -z "$line" ]`) 時，切換 `reading_first_part=false`，並跳過該行。
3. 第一部分的行直接輸出。
4. 第二部分的行用 `IFS=":" read -r first second <<< "$line"` 來分割並輸出。

這樣就能一行一行讀取檔案，直到空行，然後對接下來的行按照：分割。 🚀

📄 👍 💬 🔊

第二部分可能有未知個

如果第二部分的每一行可能有未知數量的：分隔的欄位，你可以使用 `IFS=":" read -ra parts` 來將整行切割成一個陣列。這樣就能動態處理欄位數量。

Bash Script：

```
bash
#!/bin/bash

file="your_file.txt"
reading_first_part=true # 標記是否在第一部分

while IFS= read -r line; do
    if [ -z "$line" ]; then
        reading_first_part=false
        continue # 跳過空行
    fi

    if $reading_first_part; then
        echo "First part: $line"
    else
        IFS=":" read -ra parts <<< "$line"
        echo "Second part fields:"
        for field in "${parts[@]}; do
            echo " - $field"
        done
    fi
done < "$file"
```

說明：

1. `IFS=":" read -ra parts <<< "$line"` 會將 `line` 按：分割成陣列 `parts`。
2. 用 `for field in "${parts[@]}"` 來逐一輸出陣列內的所有欄位。
3. 遇到空行後才開始處理：分割的內容。

Problem 4. Verifying an Inclusion Proof

我要怎麼讀取

leaf_index:9,tree_size:10

當中的數字呢？

使用 bash

你可以使用 `awk` 或 `grep + cut` 來提取 `leaf_index` 和 `tree_size` 的數值。以下是幾種方法：

方法 1：使用 `awk`

```
bash
echo "leaf_index:9,tree_size:10" | awk -F'[[:,]]' '{print "leaf_index="$2, "tree_size="$4}'
```

輸出：

```
leaf_index=9 tree_size=10
```

如果你只想要某一個數值，例如 `leaf_index`：

```
bash
echo "leaf_index:9,tree_size:10" | awk -F'[[:,]]' '{print $2}'
```

輸出：

```
9
```

方法 2：使用 `grep + cut`

```
bash
leaf_index=$(echo "leaf_index:9,tree_size:10" | grep -oP 'leaf_index:\K\d+')
tree_size=$(echo "leaf_index:9,tree_size:10" | grep -oP 'tree_size:\K\d+')
echo "leaf_index=$leaf_index"
echo "tree_size=$tree_size"
```

輸出：

```
leaf_index=9
tree_size=10
```

方法 3：使用 `sed`

```
bash
leaf_index=$(echo "leaf_index:9,tree_size:10" | sed -n 's/.*leaf_index:([0-9]*).*$/\1/p')
tree_size=$(echo "leaf_index:9,tree_size:10" | sed -n 's/.*tree_size:([0-9]*).*$/\1/p')
echo "leaf_index=$leaf_index"
echo "tree_size=$tree_size"
```

這些方法都可以提取數值，你可以根據需求選擇適合的方式！